

Constraint Satisfaction Problems (CSP)

Master Guide

Overview: This comprehensive guide merges the foundational logic of Google OR-Tools Constraint Programming, the core case library, and applied practice tasks into a single reference document.

(v3: Syntax Highlighting Added)

Part 1: The General Format (The Standard Recipe)

Regardless of the case, every OR-Tools program follows these 6 steps:

- 1. **Import:** `from ortools.sat.python import cp_model`
- 2. **Initialize:** `model = cp_model.CpModel()`
- 3. **Variables:** Define what the computer is deciding.
Syntax: `x = model.NewIntVar(min, max, 'name')`
- 4. **Constraints:** Define the rules.
Syntax: `model.Add(condition)` or `model.AddAllDifferent(list)`
- 5. **Solver:** Initialize and solve.
`solver = cp_model.CpSolver()`
`status = solver.Solve(model)`
- 6. **Extraction:** If `status == cp_model.FEASIBLE` or `OPTIMAL`, use `solver.Value(variable)`.

Part 2: The OR-Tools Syntax Cheat Sheet

Goal	OR-Tools Syntax
Create a 0/1 variable	<code>model.NewBoolVar('name')</code>
Create a number variable	<code>model.NewIntVar(min, max, 'name')</code>
Must be different	<code>model.AddAllDifferent([x, y, z])</code>
Must not be	<code>model.Add(x != 5)</code>
Sum of items	<code>model.Add(sum(list) == 10)</code>
Get the answer	<code>solver.Value(x)</code>

Part 3: The Case Library

Case 1: The "Unique Matching" Pattern

Description: You have N items and N slots. Every item needs a slot, and no two items can share.

The Secret Sauce: Use `model.AddAllDifferent()`.

```
from ortools.sat.python import cp_model

model = cp_model.CpModel()
workers = ['Worker_A', 'Worker_B', 'Worker_C']
num_tasks = 3

assignments = {}
for w in workers:
    assignments[w] = model.NewIntVar(1, num_tasks, w)

model.AddAllDifferent(list(assignments.values()))

solver = cp_model.CpSolver()
status = solver.Solve(model)

if status == cp_model.FEASIBLE or status == cp_model.OPTIMAL:
    print("\n" + "="*30)
    print(f"{' 📅 TASK ASSIGNMENTS':^30}")
    print("="*30)
    for w in workers:
        print(f" {w:<10} → Task {solver.Value(assignments[w])}")
    print("="*30 + "\n")
```

Case 2: The "Allowed/Forbidden" Pattern

Description: Specific items cannot go in specific spots, or can only go in specific spots.

The Approach: Use `!=` for forbidden, `==` for enforced, and `AddLinearExpressionInDomain` for multi-choice allowed.

```

from ortools.sat.python import cp_model

model = cp_model.CpModel()
vehicles = ['Truck_1', 'Truck_2', 'Truck_3']
num_routes = 3

routes = {}
for v in vehicles:
    routes[v] = model.NewIntVar(1, num_routes, v)

model.AddAllDifferent(list(routes.values()))

# Forbidden: Truck 1 != 3
model.Add(routes['Truck_1'] != 3)

# Enforced: Truck 2 == 1
model.Add(routes['Truck_2'] == 1)

# Multi-Choice Allowed: Truck 3 must be 2 or 3
model.AddLinearExpressionInDomain(routes['Truck_3'], cp_model.Domain.FromValues([2, 3]))

solver = cp_model.CpSolver()
status = solver.Solve(model)

if status == cp_model.FEASIBLE or status == cp_model.OPTIMAL:
    print("\n" + "-"*30)
    print(f"{'🚚' LOGISTICS ROUTES':^30}")
    print("-"*30)
    for v in vehicles:
        print(f" {v:<10} >> Route {solver.Value(routes[v])}")
    print("-"*30 + "\n")

```

Case 3: The "Capacity" Pattern

Description: You have a limit on a resource (Time, Money, Weight).

The Approach: Multiply a boolean variable by the cost/weight and use `sum() <= Limit`.

```
from ortools.sat.python import cp_model

model = cp_model.CpModel()

items = ['Keyboard', 'Mouse', 'Headphones', 'Webcam']
costs = [30, 15, 40, 20]
budget = 50

buy = {}
for i in range(len(items)):
    buy[i] = model.NewBoolVar(items[i])

# Constraint: The sum of (cost * decision) must be <= budget
model.Add(sum(buy[i] * costs[i] for i in range(len(items))) <= budget)

# Secondary constraint: We must buy exactly 2 items
model.Add(sum(buy[i] for i in range(len(items))) == 2)

solver = cp_model.CpSolver()
status = solver.Solve(model)

if status == cp_model.FEASIBLE or status == cp_model.OPTIMAL:
    total_spent = sum(solver.Value(buy[i]) * costs[i] for i in range(len(items)))

    print("\n🛒 SHOPPING CART:")
    print("="*25)
    for i in range(len(items)):
        if solver.Value(buy[i]) == 1:
            print(f"  [+] {items[i]:<12} ${costs[i]}")
    print("-" * 25)
    print(f"  TOTAL SPENT:      ${total_spent} / ${budget}\n")
```

Case 4: The "Logical Link" Pattern

Description: One decision forces or forbids another decision.

The Approach: Use `.OnlyEnforceIf()`. (Note: OnlyEnforceIf requires boolean variables).

```

from ortools.sat.python import cp_model

model = cp_model.CpModel()
devs = ['Dev_A', 'Dev_B', 'Dev_C', 'Dev_D']

team = {}
for d in devs:
    team[d] = model.NewBoolVar(d)

# "If A is selected, then enforce that B is selected"
model.Add(team['Dev_B'] == 1).OnlyEnforceIf(team['Dev_A'])

# "If C is selected, then enforce that D is NOT selected"
model.Add(team['Dev_D'] == 0).OnlyEnforceIf(team['Dev_C'])

# Force A and C to be picked for testing
model.Add(team['Dev_A'] == 1)
model.Add(team['Dev_C'] == 1)

solver = cp_model.CpSolver()
status = solver.Solve(model)

if status == cp_model.FEASIBLE or status == cp_model.OPTIMAL:
    print("\n🧑‍💻 DEVELOPER TEAM ROSTER")
    print("-" * 25)
    for d in devs:
        if solver.Value(team[d]) == 1:
            print(f"✅ {d}")
        else:
            print(f"❌ {d}")
    print("-" * 25 + "\n")

```

Part 4: Applied Practice Tasks

Task 2: 4x4 Sudoku Solver

Rules: Each cell contains a value from {1,2,3,4}. Each row, column, and 2x2 sub-grid must contain all numbers exactly once.

Given partially filled grid:

0	0	2	0
0	0	0	3
0	3	0	0
1	0	0	0

```

from ortools.sat.python import cp_model

model = cp_model.CpModel()
size = 4

# 1. Variables
grid = {}
for r in range(size):
    for c in range(size):
        grid[(r, c)] = model.NewIntVar(1, 4, f'cell_{r}_{c}')

# 2. Case 2: Enforced Values (The starting grid)
initial_grid = [
    [0, 0, 2, 0],
    [0, 0, 0, 3],
    [0, 3, 0, 0],
    [1, 0, 0, 0]
]

for r in range(size):
    for c in range(size):
        if initial_grid[r][c] != 0:
            model.Add(grid[(r, c)] == initial_grid[r][c])

# 3. Case 1: Unique Matching (Rows and Columns)
for i in range(size):
    model.AddAllDifferent([grid[(i, c)] for c in range(size)])
    model.AddAllDifferent([grid[(r, i)] for r in range(size)])

# 4. Case 1: Unique Matching (2x2 Sub-grids)
for r in (0, 2):
    for c in (0, 2):
        sub_grid = []
        for di in range(2):
            for dj in range(2):
                sub_grid.append(grid[(r + di, c + dj)])
        model.AddAllDifferent(sub_grid)

# Solve and Pretty Print
solver = cp_model.CpSolver()
status = solver.Solve(model)

if status == cp_model.FEASIBLE or status == cp_model.OPTIMAL:
    print("\n🍀 4x4 SUDOKU SOLUTION 🍀")
    print("┌───┐")
    for r in range(size):
        if r == 2:
            print("├───┼───┼───┤")

        # Extract row values
        row = [str(solver.Value(grid[(r, c)])) for c in range(size)]

        # Format with dividers
        print(f"| {row[0]} | {row[1]} | {row[2]} | {row[3]} |")
    print("└───┴───┴───┘\n")

```

Task 3: Exam Scheduling with Optimization

Rules & Optimization: There are 5 exams and 3 available time slots. Conflicting exams cannot be scheduled at the same time. Each exam must be assigned exactly one time slot. **Requirement:** Minimize the number of late-slot exams (slot 3 considered late).

Conflicts: Exam 1 conflicts with Exam 2. Exam 2 conflicts with Exam 3. Exam 4 conflicts with Exam 5.


```

from ortools.sat.python import cp_model

model = cp_model.CpModel()
num_exams = 5
num_slots = 3

# 1. Variables
exams = {}
for i in range(1, num_exams + 1):
    exams[i] = model.NewIntVar(1, num_slots, f'exam_{i}')

# 2. Case 2: Forbidden (Conflict constraints)
model.Add(exams[1] != exams[2])
model.Add(exams[2] != exams[3])
model.Add(exams[4] != exams[5])

# 3. Case 4: Logical Links (Detecting "Late" Exams)
is_late = {}
for i in range(1, num_exams + 1):
    is_late[i] = model.NewBoolVar(f'is_late_{i}')

    # If is_late == 1, then the exam MUST be in slot 3
    model.Add(exams[i] == 3).OnlyEnforceIf(is_late[i])

    # If is_late == 0, then the exam MUST be strictly less than 3
    model.Add(exams[i] < 3).OnlyEnforceIf(is_late[i].Not())

# 4. Optimization: Minimize the total number of late exams
model.Minimize(sum(is_late[i] for i in range(1, num_exams + 1)))

# Solve and Pretty Print
solver = cp_model.CpSolver()
status = solver.Solve(model)

if status == cp_model.FEASIBLE or status == cp_model.OPTIMAL:
    print("\n" + "="*38)
    print(f"{'🏆' OPTIMAL EXAM SCHEDULE':^38}")
    print("="*38)
    print(f"| {'Exam':<6} | {'Slot':<6} | {'Status':<13} |")
    print("-" * 38)

    for i in range(1, num_exams + 1):
        slot = solver.Value(exams[i])
        status_txt = "⚠️ LATE" if slot == 3 else "✅ ON TIME"
        print(f"| Exam {i:<4} | Slot {slot:<4} | {status_txt:<13} |")

    print("="*38)
    print(f"Total Late Exams (Slot 3): {int(solver.ObjectiveValue())}\n")

```

Adversarial Search

Master Guide

Overview: This master guide covers Game Theory mechanics, the Minimax Algorithm, and Alpha-Beta Pruning. It contains the core definitions, pseudo-code cheat sheets, and the final Python implementations using dark-mode syntax highlighting.

Part 1: The Core Elements (Game Theory)

Adversarial Search models competitive environments (multiagent) where players have conflicting goals (e.g., "I Win" vs. "You Lose"). A zero-sum game assumes no luck factor.

Element	Definition
Initial State (So)	The top node of the game tree representing the starting point of the game.
Player(s)	Defines whose turn it is. Usually Max (maximizes chances of winning) and Min (minimizes Max's chances).
Actions / Moves	The set of legal moves available to a player in a specific state.
Result(s, a)	A transition model describing the outcome of a specific move.
Terminal Test	Checks if the game has ended (e.g., all boxes filled, checkmate).
Utility Function	The objective/payoff function returning the final value (e.g., +1 for win, -1 for loss, 0 for draw).

Part 2: Algorithm Cheat Sheet

1. Minimax Strategy

Follows Depth-First Search (DFS) to propagate utility values from leaf nodes to the root. Assumes perfect play from both the Max and Min players.

- **Max's Turn:** Accepts the MAXIMUM value from its successor nodes.
- **Min's Turn:** Accepts the MINIMUM value from its successor nodes.

2. Alpha-Beta Pruning

An optimized version of Minimax. It eliminates ("prunes") branches in the search tree that cannot possibly influence the final decision, preventing unnecessary calculations.

- **Alpha (α):** The best (highest) value that the Max player can guarantee. Starts at $-\infty$.
- **Beta (β):** The best (lowest) value that the Min player can guarantee. Starts at $+\infty$.
- **Pruning Condition:** Stop exploring a branch if $\beta \leq \alpha$.

Part 3: Python Implementations

Case 1: Standard Minimax Implementation

Standard implementation using recursive DFS. Generates the full game tree without skipping any nodes.

```
import math

class Node:
    def __init__(self, value=None):
        self.value = value
        self.children = []
        self.minmax_value = None

class Environment:
    def __init__(self, tree):
        self.tree = tree
        self.computed_nodes = []

    def compute_minimax(self, node, depth, maximizing_player=True):
        # Base case: terminal node or depth limit reached
        if depth == 0 or not node.children:
            self.computed_nodes.append(node.value)
            return node.value

        if maximizing_player:
            value = -math.inf
            for child in node.children:
                child_value = self.compute_minimax(child, depth - 1, False)
                value = max(value, child_value)
            node.minmax_value = value
            self.computed_nodes.append(node.value)
            return value

        else: # Minimizing player
            value = math.inf
            for child in node.children:
                child_value = self.compute_minimax(child, depth - 1, True)
                value = min(value, child_value)
            node.minmax_value = value
            self.computed_nodes.append(node.value)
            return value
```

Case 2: Alpha-Beta Pruning Implementation

Optimized version tracking Alpha and Beta thresholds. If a path is determined to be worse than a previously explored path, it breaks the loop (prunes the branch).

```
import math

class Node:
    def __init__(self, value=None):
        self.value = value
        self.children = []
        self.minmax_value = None

class Environment:
    def __init__(self, tree):
        self.tree = tree
        self.computed_nodes = []

    def alpha_beta_search(self, node, depth, alpha, beta, maximizing_player=True):
        self.computed_nodes.append(node.value)

        # Base case: terminal node or depth limit
        if depth == 0 or not node.children:
            return node.value

        if maximizing_player:
            value = -math.inf
            for child in node.children:
                value = max(value, self.alpha_beta_search(child, depth - 1, alpha, beta,
False))
                alpha = max(alpha, value)

            # Pruning condition
            if beta <= alpha:
                print(f"✂ Pruned node: {child.value}")
                break
            node.minmax_value = value
            return value

        else: # Minimizing player
            value = math.inf
            for child in node.children:
                value = min(value, self.alpha_beta_search(child, depth - 1, alpha, beta,
True))
                beta = min(beta, value)

            # Pruning condition
            if beta <= alpha:
                print(f"✂ Pruned node: {child.value}")
                break
            node.minmax_value = value
            return value
```

Supervised Learning

Master Guide (Case Library)

Overview: This guide categorizes the core algorithms from your Supervised Learning curriculum into three distinct "Cases." Memorize the recipe for each case to instantly know how to process data, train the model, and evaluate it during an exam.

Part 1: The General ML Recipe

Every supervised learning problem follows this 5-step pipeline:

- 1. **Load & Clean:** Handle missing values (`.fillna()` or `.dropna()`).
- 2. **Encode Categoricals:** Convert text to numbers (`LabelEncoder` , `.map()` , or `pd.get_dummies`).
- 3. **Split Data:** Separate features (`X`) from target (`y`), then use `train_test_split` .
- 4. **Scale (If Needed):** Apply `StandardScaler` (crucial for distance-based models like SVM).
- 5. **Train & Evaluate:** Fit the model, predict on test data, and check metrics.

Part 2: Evaluation Metrics Cheat Sheet

Problem Type	Metric	When to Use It
Regression	MAE / RMSE	Tells you how far off your predictions are on average. Lower is better.
Regression	R2 Score	Explains how much variance is captured. Closer to 1.0 is perfect.
Classification	Accuracy	Overall correct predictions. Good for balanced datasets.
Classification	Precision & Recall	Use when false positives or false negatives are costly (e.g., medical testing or fraud).
Classification	F1-Score	The harmonic mean of Precision and Recall. Best for imbalanced datasets.

Part 3: The Case Library

Case 1: The "Continuous Outcome" Pattern (Linear Regression)

Scenario: You need to predict an exact numerical value. (e.g., *Student Final Exam Score*, *House Price*, *Salary*).

The Approach: Clean missing numbers using mean/median. Use `LinearRegression`. Evaluate with Error Metrics (MAE/RMSE/R2).

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import pandas as pd
import numpy as np

# --- Case 1: Continuous Outcome (Regression) ---
# Goal: Predict a numerical value (e.g., student final score)

df = pd.read_csv('dataset.csv')

# 1. Preprocessing: Fill missing numeric values with mean/median
df['study_hours'] = df['study_hours'].fillna(df['study_hours'].mean())

# 2. Preprocessing: Ordinal Encoding for ranked categories
df['participation'] = df['participation'].map({'Low': 0, 'Medium': 1, 'High': 2})

X = df[['study_hours', 'participation']]
y = df['final_score']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. Model Training
model = LinearRegression()
model.fit(X_train, y_train)

# 4. Evaluation
predictions = model.predict(X_test)
print("MAE:", mean_absolute_error(y_test, predictions))
print("RMSE:", np.sqrt(mean_squared_error(y_test, predictions)))
print("R2 Score:", r2_score(y_test, predictions))
```

Case 2: The "Rule-Based Classification" Pattern (Decision Tree)

Scenario: You need to categorize data, and you want to understand the "if-then" rules behind the decision. (e.g., *Loan Approval*, *Disease Diagnosis*).

The Approach: Use `LabelEncoder` for text columns. Feature scaling is **not** required for Decision Trees. Evaluate with Accuracy and Classification Report.

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report
from sklearn.preprocessing import LabelEncoder
import pandas as pd

# --- Case 2: Rule-Based Classification (Decision Tree) ---
# Goal: Predict a discrete category (e.g., Loan Approved: 1 or 0)

df = pd.read_csv('dataset.csv')
df = df.dropna() # Quick drop of missing values

# 1. Preprocessing: Label Encoding for binary/simple text classes
le = LabelEncoder()
df['employment_status'] = le.fit_transform(df['employment_status'])

X = df[['income', 'employment_status', 'credit_score']]
y = df['loan_approved']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# 2. Model Training
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)

# 3. Evaluation
y_pred = dt_model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```


Case 3: The "Boundary Classification" Pattern (SVM)

Scenario: You need to divide complex data into categories by drawing an optimal line or hyperplane. (e.g., *Customer Churn Prediction*, *Image Classification*).

The Approach: Use One-Hot Encoding (`get_dummies`). You **MUST** use `StandardScaler` before training an SVM, otherwise, it will fail to find the correct boundary. Evaluate with Confusion Matrix.

```
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix, classification_report
import pandas as pd

# --- Case 3: Boundary Classification (Support Vector Machine) ---
# Goal: Find optimal hyperplane to separate classes (e.g., Customer Churn)

df = pd.read_csv('dataset.csv')
df = df.dropna()

# 1. Preprocessing: One-Hot Encoding for nominal (non-ranked) text categories
df = pd.get_dummies(df, columns=['contract_type'], drop_first=True)

X = df.drop('churn', axis=1)
y = df['churn']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 2. Preprocessing: CRITICAL STEP - Feature Scaling for SVM
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 3. Model Training
svm_model = SVC(kernel='linear', random_state=42)
svm_model.fit(X_train_scaled, y_train)

# 4. Evaluation
y_pred = svm_model.predict(X_test_scaled)
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Unsupervised Learning

Master Guide (Case Library)

Overview: This guide covers K-Means Clustering based on Lab 11. Unlike Supervised Learning, there are no "correct answers" (labels). We group data based on similarity using distance.

Part 1: The K-Means Workflow

1. **Choose K:** Use the Elbow Method (WCSS) to find the "bend" in the graph.
2. **Initialize:** Select K random points as initial centroids (usually `init='k-means++'`).
3. **Assign:** Assign each data point to the nearest centroid.
4. **Update:** Calculate the mean of points in each cluster and move the centroid there.
5. **Repeat:** Continue until centroids stop moving.

Part 2: The Case Library

Case 1: The "Elbow" Pattern (Finding Optimal K)

Scenario: You are given a dataset but don't know how many groups exist.

The Approach: Loop through K (1 to 10), calculate `inertia_` (WCSS), and plot it.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# --- Case 1: The Elbow Method (Finding K) ---
# Goal: Determine the optimal number of clusters

data = pd.read_csv('data.csv')
X = data[['feature1', 'feature2']] # Select relevant features

wcss = []
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, init='k-means++', random_state=42)
    kmeans.fit(X)
    wcss.append(kmeans.inertia_)

# Plotting the Elbow Curve
plt.plot(range(1, 11), wcss, marker='o')
plt.title('The Elbow Method')
plt.xlabel('Number of clusters (k)')
plt.ylabel('WCSS')
plt.show()
```

Case 2: The "Scaled Clustering" Pattern (Task 1, 2, & 3)

Scenario: Clustering features with different units (e.g., Age 0-100 vs. Salary 0-100,000).

The Approach: You **MUST** use `StandardScaler`. Without it, the feature with the largest numbers will dominate the distance calculation.

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
import pandas as pd

# --- Case 2: K-Means with Scaling (Standard Pipeline) ---
# Goal: Cluster data where features have different scales (e.g., GPA vs Mileage)

df = pd.read_csv('students.csv')

# 1. Feature Selection
X = df[['GPA', 'study_hours', 'attendance_rate']]

# 2. Scaling (Crucial for K-Means as it uses Euclidean Distance)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3. Fit K-Means
kmeans = KMeans(n_clusters=3, init='k-means++', random_state=42)
df['cluster'] = kmeans.fit_predict(X_scaled)
```

```
# 4. Output results
print(df[['student_id', 'cluster']])
```

Case 3: The "Visualization" Pattern

Scenario: You need to present the clusters and centroids visually.

The Approach: Use `plt.scatter` with color filtering based on the cluster labels.

```
import matplotlib.pyplot as plt

# --- Case 3: Cluster Visualization ---
# Goal: Visualize 2D clusters and their centers

# Assuming y_predict contains cluster labels and X is your data array
plt.scatter(X[y_predict == 0, 0], X[y_predict == 0, 1], s=100, c='red', label='Cluster 1')
plt.scatter(X[y_predict == 1, 0], X[y_predict == 1, 1], s=100, c='blue', label='Cluster 2')
plt.scatter(X[y_predict == 2, 0], X[y_predict == 2, 1], s=100, c='green', label='Cluster 3')

# Plotting Centroids
plt.scatter(kmeans.cluster_centers[:, 0], kmeans.cluster_centers[:, 1], s=300, c='yellow', label='Centroids')

plt.title('Clusters of Students')
plt.xlabel('Study Hours')
plt.ylabel('GPA')
plt.legend()
plt.show()
```

Exploratory Data Analysis

Master Guide (EDA Library)

Overview: EDA is the "storytelling" phase of data science. Before building models, you must find patterns, detect outliers, and understand correlations using visual methods.

Part 1: The Visual Toolbox

Plot Type	Usage	Library
Histogram / Distplot	Shows distribution & skewness of a single variable.	Seaborn / Matplotlib
Box Plot	Best for identifying Outliers (dots outside the whiskers).	Seaborn
Heatmap	Visualizes the Correlation Matrix between features.	Seaborn
Scatter Plot	Shows the relationship/trend between two variables.	Seaborn

Part 2: The EDA Case Library

Case 1: Data Inspection (The "First Look")

Scenario: You just loaded a CSV and need to see what's broken.

The Approach: Use `.info()` and `.isnull()` to find missing data.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# --- Case 1: Data Inspection & Cleaning ---
# Goal: Understand structure and fix missing values

df = pd.read_csv('dataset.csv')

# 1. Quick Inspection
print(df.info())      # Column names, types, and non-null counts
print(df.describe())  # Statistical summary (mean, std, min, max)
print(df.isnull().sum()) # Count of missing values per column

# 2. Filter Numerical Columns Only
df_num = df.select_dtypes(include=['float64', 'int64'])

# 3. Handling Missing Data (Universal methods)
df['column'] = df['column'].fillna(df['column'].median()) # Numerical
df['cat_col'] = df['cat_col'].fillna(df['cat_col'].mode()[0]) # Categorical
```

Case 2: Distribution & Outliers (Univariate)

Scenario: You need to see if your target (e.g., SalePrice) is normally distributed or has extreme values.

The Approach: Plot a Histogram and a Box Plot.

```
import seaborn as sns
import matplotlib.pyplot as plt

# --- Case 2: Univariate Analysis (Distribution) ---
# Goal: See how a single variable is spread out (Check for Skew/Outliers)

# 1. Histogram / Distplot
plt.figure(figsize=(9, 8))
sns.histplot(df['SalePrice'], color='g', bins=100, kde=True)
plt.title('Distribution of Price')
plt.show()

# 2. Box Plot (Best for detecting outliers)
sns.boxplot(x=df['SalePrice'])
plt.title('Detecting Outliers in Price')
plt.show()
```

Case 3: Relationships & Correlation (Multivariate)

Scenario: You need to identify which features affect your target the most.

The Approach: Generate a correlation matrix and visualize it with a Heatmap.

```
import seaborn as sns
import matplotlib.pyplot as plt

# --- Case 3: Multivariate Analysis (Correlation) ---
# Goal: Find relationships between variables

# 1. Correlation Matrix
corr = df_num.corr()

# 2. Heatmap Visualization
plt.figure(figsize=(12, 10))
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Feature Correlation Heatmap')
plt.show()

# 3. Scatter Plot (Relationship between two specific features)
sns.scatterplot(data=df, x='GrLivArea', y='SalePrice')
plt.title('Living Area vs Sale Price')
plt.show()
```